

Shift Schedule Optimization

Tobias Michlik

May 13, 2026

Intro

We want to find the best schedule given everyone's shift availability and preference. The result has to be valid and optimized. A valid schedule follows the following rules:

1. Anyone working a shift must work its full number of hours
2. All required shifts are filled with the right number of workers,
3. No worker is working a shift they aren't available for, and
4. Each shift is wholly worked by one worker. Some shifts are worth more than 1 working hour and cannot be split between workers.

Each worker gives a preference score in $[1, 10]$ for the shifts they are available for (and 0 for unavailable shifts). An optimized shift maximizes the sum of preference scores.

Variables

Consider the following variables:

- n = number of workers
- k = number of shifts
- $P \in \mathbb{Z}^{n \times k}$ represents worker availability and preference data:

$\forall 1 \leq i \leq n$ and $\forall 1 \leq j \leq k$, let $0 \leq P_{ij} \leq 10$ represent worker i 's preference score for shift j . 0 indicates a shift they aren't available for.

Example:

$$P^1 = \begin{bmatrix} 0 & 0 & 1 & 10 \\ 2 & 0 & 3 & 8 \\ 4 & 4 & 9 & 0 \\ 0 & 0 & 6 & 0 \\ 9 & 8 & 4 & 9 \end{bmatrix}$$

- $S \in \mathbb{Z}^{k \times 2}$ represents shift data:

$\forall 1 \leq m \leq k$, S_{m1} = shift working hours for shift m
 S_{m2} = number of workers for shift m

Example:

$$S^1 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \\ 1 & 1 \\ 2 & 1 \end{bmatrix}$$

- $x = \max$ number of hours per worker =

$$\max \left(\left\lceil \frac{1}{n} \sum_{m=1}^k S_{m1} \cdot S_{m2} \right\rceil, \max_{1 \leq m \leq k} S_{m1} \right)$$

In our example, let $n_1 = 5$:

$$x_1 = \left\lceil \frac{1}{n_1} \sum_{m=1}^{k_1} S_{m1}^1 \cdot S_{m2}^1 \right\rceil = \max_{1 \leq m \leq k_1} S_{m1}^1 = 2$$

Objective Function

- $\theta \in \mathbb{R}^{n \times x}$ is the input to our objective function. Let $\Theta \in \mathbb{Z}^{n \times x}$ be $\text{round}(\theta)$ (each element of θ is rounded to the nearest integer).

$\forall 1 \leq i \leq n$ and $1 \leq j \leq x$, $0 \leq \theta_{ij} \leq k$. We get our solution by rounding each θ_{ij} to the nearest integer to get worker i 's j th assigned shift with index of $\text{round}(\theta_{ij})$. A zero represents no scheduled shift.

Example:

$$\theta^1 = \begin{bmatrix} 4.0 & 3.9 \\ 0.0 & 0.3 \\ 1.9 & 0.1 \\ 3.2 & 0.0 \\ 1.3 & 1.2 \end{bmatrix} \implies \text{round}(\theta^1) = \Theta^1 = \begin{bmatrix} 4 & 4 \\ 0 & 0 \\ 2 & 0 \\ 3 & 0 \\ 1 & 1 \end{bmatrix}, \text{ a valid solution.}$$

- $\forall 1 \leq i \leq n$, $\hat{P}_i : [0, k] \rightarrow \mathbb{R}$ is a helper function to the objective function.

$$\hat{P}_i(s) = (1 - \alpha)P_{i\lfloor s \rfloor} + \alpha P_{i\lceil s \rceil}, \quad \alpha = s - \lfloor s \rfloor$$

- $f(\theta) : \mathbb{R}^{n \times x} \rightarrow \mathbb{R}$ is our objective function. We aim to minimize the objective function.

$$f(\theta) = - \sum_{i=1}^n \sum_{j=1}^x \hat{P}_i(\theta_{ij})$$

Constraints

1. Every shift m must appear exactly S_{m1} times across the entire Θ matrix. With $\mathbf{1}$ as an indicator function:

$$\forall 1 \leq m \leq k, \quad S_{m1} = \sum_{i=1}^n \sum_{j=1}^x \mathbf{1}(\Theta_{ij} = m) \quad (\text{C1})$$

2. Every shift m must be found in S_{m2} unique rows within Θ :

$$\forall 1 \leq m \leq k, \quad S_{m2} = \sum_{i=1}^n \mathbf{1} \left(\sum_{j=1}^x \mathbf{1}(\Theta_{ij} = m) \geq 1 \right) \quad (\text{C2})$$

3. No worker can be assigned a shift where their preference is 0/they are unavailable:

$$\forall i, j, \quad \hat{P}_i(\theta_{ij}) > 0 \quad (\text{C3})$$

4. If any part of a shift m is assigned to a worker i , then the entire length S_{m1} must be assigned to that same worker i :

$$\forall 1 \leq m \leq k, \quad \exists 1 \leq i \leq n \text{ s.t. } S_{m1} = \sum_{j=1}^x \mathbf{1}(\Theta_{ij} = m) \quad (\text{C4})$$

Initial NLP

$$(Q) \quad \left\{ \begin{array}{ll} \underset{\theta \in \mathbb{R}^{n \times x}}{\text{minimize}} & f(\theta) \\ \text{subject to} & S_{m1} = \sum_{i=1}^n \sum_{j=1}^x \mathbf{1}(\Theta_{ij} = m) \quad \forall 1 \leq m \leq k \\ & S_{m2} = \sum_{i=1}^n \mathbf{1} \left(\sum_{j=1}^x \mathbf{1}(\Theta_{ij} = m) \geq 1 \right) \quad \forall m \\ & \hat{P}_i(\theta_{ij}) > 0 \quad \forall i, j \\ & \exists 1 \leq i \leq n \text{ s.t. } S_{m1} = \sum_{j=1}^x \mathbf{1}(\Theta_{ij} = m) \quad \forall m \end{array} \right.$$

Ideally, we optimize a NLP without any constraints. Let's consider ways to optimize this.

Optimizing Constraints

Now, we should convert our constraints into penalties.

Penalty_{C1} Indicator functions aren't very good for generating continuous penalties. Our first step is to stop checking when $\Theta_{ij} = m$ and instead determine how far $\theta_{ij} - m$ is from 0. So, for each shift, instead of checking our solution has the right count of working hours for the shift, we check the sum of a continuous analogue to our indicator function. The function we'll use is kernel density estimator $K : \mathbb{R}^3 \rightarrow \mathbb{R}$

$$K_\sigma(\phi) = \exp \left(-\frac{(\phi)^2}{2\sigma^2} \right).$$

Think of it as a bell curve around with mean 0 and variance σ^2 . When optimizing:

- Start with a wide σ .

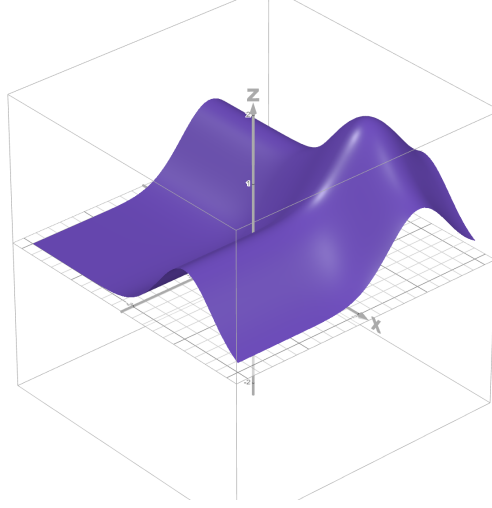


Figure 1: C_3^1 across all valid inputs shifted toward origin by 2 points

- Gradually ramp it down.
- Eventually it'll force the optimizer to output near integers within θ .

Now, for every shift, m , we sum up these densities across every assigned shift in θ :

$$C_m(\theta) = \sum_{i=1}^n \sum_{j=1}^x K_\sigma(\theta_{ij} - m).$$

With $m = 3$ and $\sigma = 0.5$, C_3^1 across all possible θ values is a pointed sheet with two perpendicular ridges, as shown in Figure 1

Since we now have a distribution of shift coverage, for each shift, we have to test if it's been covered properly. To do so, we compare the difference between the coverage score and shift working hours:

$$\text{Penalty}_{C1} = \sum_{m=1}^k (C_m(\theta) - (S_{m1} \cdot S_{m2}))^2.$$

If a shift is over- or under-assigned, there is a positive penalty.

Penalty_{C2} Since a worker can only work 1 role in a shift, they cannot exceed working the base shift duration. Thus, we penalize whenever a worker's work duration for a shift exceeds the shift duration. Consider the shift coverage score *per worker*:

$$C_{i,m}(\theta) = \sum_{j=1}^x K_\sigma(\theta_{ij} - m).$$

We apply a hinge loss penalty:

$$\text{Penalty}_{C2} = \sum_{m=1}^k \sum_{i=1}^n \max(0, C_{i,m}(\theta) - S_{m1})^2$$

Penalty_{C3} With C3 as $\hat{P}_i(\theta_{ij}) > 0$, we need a big penalty as $\hat{P}_i$ gets near 0. To do so, we apply a hinge loss, penalizing when preference score falls below τ :

$$\text{Penalty}_{C3} = \sum_{i=1}^n \sum_{j=1}^x \max(0, \tau - \hat{P}_i(\theta_{ij}))^2, \quad \tau = 1$$

In this case, τ can be set to 1 since we don't want to be assigning unavailable shifts which have preference score below 1.

Penalty_{C4} We can do a very similar thing for $C4$ as for $C1$. The difference is that we're primarily checking each worker covers the full hours of their assigned shift. We need to penalize whenever a worker has neither the full shift (S_{m1}) nor none of it (0).

Using the same square error, we can penalize deviance from a worker having less parts of a shift. The square of the coverage score penalizes having the shift. These together makes the desired effect.

$$\text{Penalty}_{C4} = \sum_{m=1}^k \sum_{i=1}^n [C_{i,m}(\theta)^2 \cdot (C_{i,m}(\theta) - S_{m1})^2].$$

Total Penalty When we put out penalties together, we use 4 hyperparameters to control the strength of the corresponding penalty terms:

$$\lambda_1 \text{Penalty}_{C1} + \lambda_2 \text{Penalty}_{C2} + \lambda_3 \text{Penalty}_{C3} + \lambda_4 \text{Penalty}_{C4}.$$

Tuning these is a bit of a task, but here are included some techniques varying in sophistication. And don't forget to tune σ (potentially distinct within Penalty_{C1} and Penalty_{C4}).

1. Grid Search:

- We select $\lambda_i = 10^\alpha$, $\alpha \in [-4, 4]$.
- Run the optimization for each combination
- Evaluate preference score, scheduling violations, and atomic coverage error.
- Select best λ s.

2. Better yet is to treat λ s as hyperpriors in a Bayesian model. We put priors on λ s and infer them by sampling.

3. Adaptive Tuning: Start with small penalties and

- increase them as optimization progresses,

$$\lambda_i^{(t+1)} = \gamma \lambda_i^{(t)}, \quad \gamma > 1,$$

or

- update them based on constraint violations,

$$\lambda_i^{(t+1)} = \lambda_i^{(t)} + \rho \cdot \text{violation}, \quad \rho > 1.$$

Final NLP

Since we have converted our constraints all into penalties, we can write our final objective function as a single continuous and piecewise differentiable function.

$$(Q) \quad \left\{ \begin{array}{l} \text{minimize} \\ \theta \in \mathbb{R}^{n \times x} \end{array} \right. f(\theta) + \lambda_1 \text{Penalty}_{C1} + \lambda_2 \text{Penalty}_{C2} + \lambda_3 \text{Penalty}_{C3} + \lambda_4 \text{Penalty}_{C4}$$